

Labyrinth Breach

Reinforcement Learning-based Multi-Agent Pursuit-Evasion in Dynamic Maze Environments

Muhammad Sikander Raheem • Usman Irshad Bhatti • Ahmed Jawed

MS AI, LUMS | AI641 – AI for Robotics

Unity ML-Agents | PPO | MARL

Project Overview

What is Labyrinth Breach?

Environment: Unity ML-Agents maze simulator

Setup: 3 Sentinels vs 2 Runners (3v2 asymmetric)

Goal (Sentinels): Capture both Runners before timeout

Goal (Runners): Reach an exit zone or survive 120s

Mazes: Procedurally generated with dynamic wall shifts

Key Technical Challenges



Partial Observability

Raycasts + last-known-position memory; no global state



Multi-Agent Coordination

3 agents must cooperate to trap 2 evaders



Dynamic Environment

Walls shift mid-episode, forcing real-time adaptation



Asymmetric Teams

Different team sizes, roles, reward structures



Generalization

Must work on unseen maze layouts

Literature Review

Foundations & Related Work

Pursuit-Evasion

Cooperative MARL

Credit Assignment

Partial Observability

Theme 1 – Pursuit-Evasion as a Multi-Agent Problem

Core Concept

Classical robotics & MARL problem: one side captures, the other evades. RL is a natural fit because the desired behavior is hard to hand-code under adversarial interaction, obstacles, partial information, and multiple cooperating agents.

Richer Behaviors Required

Sentinel Behaviors:

- Trapping & pincer movement
- Corridor denial & exit blocking
- Route cutting

Runner Behaviors:

- Distance management & path selection
- Survival & timing of escape
- Exploit topological openings

Key References

Verma et al. (2019)

Multi-agent pursuer-evader learning using Situation Report; emergent route-blocking from simple rewards.

Yan et al. (2019)

Pursuit-evasion with uncertain pursuer speeds; coordination under real-world constraints.

Liu et al. (2024)

Multi-UAV pursuit-evasion via PSO-M3DDPG; large-scale aerial coordination.

Haeri et al. (2022)

Emergent behaviors in multi-agent target acquisition; rich behavior from simple reward signals.

Theme 2 – Cooperative & Mixed Multi-Agent RL

Core Challenge: Non-Stationarity

All agents update simultaneously — each agent's environment is non-stationary from the perspective of any single agent, making standard RL convergence guarantees inapplicable.

MADDPG [Lowe et al., 2017]

Centralized Training, Decentralized Execution (CTDE): each actor uses only local observations at test time, but critic accesses global state during training. Seminal solution to non-stationarity.

Our use: Inspired our CTDE framing; we use PPO's parameter sharing as a simpler alternative.

PPO in Cooperative Games [Yu et al., 2022]

Showed that well-tuned PPO baselines surprisingly outperform more complex algorithms in cooperative multi-agent settings. Lower complexity, better reproducibility.

Our use: Directly adopted as our primary training method; Unity ML-Agents provides native PPO support.

Parameter Sharing

Agents within a team share a single policy network. Reduces sample complexity and enforces role symmetry. Standard in homogeneous cooperative teams.

Our use: All three Sentinels share one policy; both Runners share another — distinct from each other.

Theme 3 – Credit Assignment in Cooperative MARL

The Credit Assignment Problem

When multiple agents cooperate under team-level objectives, one agent may contribute by blocking a route or creating pressure while another gets the final capture.

Plain shared reward is too coarse — agents cannot distinguish meaningful individual contributions from free-riding.

This problem intensifies in sparse-reward environments where terminal win/loss signals may not arrive for hundreds of steps.

Our Approach

Tactical reward shaping with per-agent bonuses for coordination events: pincer formations, corridor control, exit denial, and dead-end forcing.

Landmark Solutions

QMIX [Rashid et al., 2018]

Value decomposition

Monotonic value function factorisation: global Q-value is a monotonic mix of individual agent Q-values. Guarantees that the decentralised argmax equals the centralised argmax.

Counterfactual PG [Foerster et al., 2018]

Gradient-based

Baseline subtracts the counterfactual contribution of each agent, providing a cleaner gradient signal that isolates individual impact from team outcomes.

Implicit Credit Assign. [Wang et al., 2020]

Attention-based

Agents implicitly learn to assign credit through attention mechanisms over teammates' actions, without explicit handcrafted reward splits.

Polarization PG [Chen et al., 2023]

Explicit

Explicit credit via polarization of policy gradients — reinforces cooperative agents while penalizing free-riders within the team.

Theme 4 – Partial Observability in Pursuit-Evasion

Why it matters:

With full global state, pursuit-evasion becomes a trivial tracking problem. Partial observability forces agents to plan under uncertainty, maintain beliefs about hidden opponents, and use memory — making the task far more realistic and challenging.

ViPER [Wang, Sartoretti & How, 2025]

Visibility-based Pursuit-Evasion via RL. Agents observe only within a line-of-sight cone. The evader exploits blind-spots; the pursuer must sweep and track.

Key insight: line-of-sight constraints fundamentally change optimal strategies for both sides.

vs. Labyrinth Breach: Labyrinth Breach adds team coordination pressure through the 3v2 asymmetry and exit-based objectives that ViPER lacks.

Gonultas & Isler [2024]

Pursuit-evasion with dynamic sensor and actuation constraints. Explores how limited range, noise, and uncertain speeds change optimal pursuit strategy under RL.

Key insight: uncertainty about opponent dynamics is as important as sensing range.

vs. Labyrinth Breach: Our memory-augmented observation (last-known-position) addresses the same uncertainty at a system level, without explicit sensor modelling.

Synthesis – How the Literature Shapes Our Design

Theme	Key Insight	Design Decision	Refs
Pursuit-Evasion	Rich behaviors only emerge in structured, topologically complex environments	Procedural maze + exit zones + dynamic walls	[5][8][10][11][16]
Cooperative MARL	PPO with parameter sharing is a competitive baseline for cooperative settings	Shared policy per team, PPO training via Unity ML-Agents	[4]
CTDE	Centralised critic / decentralised actor resolves non-stationarity	MA-POCA supported in config; PPO used in reported experiments	[2]
Credit Assignment	Shared reward too coarse for indirect contributions (e.g. blocking)	Tactical shaping: pincer, corridor control, exit denial, dead-end forcing	[3][6][7][12]
Partial Observability	Line-of-sight + memory changes strategy fundamentally	Raycasts + last-known-position; no global state; no camera images	[8][9][10]

Problem formulation

We model the system as a Partially Observable Stochastic Game with five agents; three Sentinels and two Runners. Formally it's the tuple $\langle N, S, \{A_i\}, \{O_i\}, T, \{R_i\}, \gamma \rangle$ where $N=5$, the joint state covers positions, velocities, and wall configuration, the transition function is given by Unity's physics engine, and $\gamma=0.99$.

The interaction is asymmetric and team-versus-team: Sentinels win when both Runners are captured before timeout; Runners win when one of them reaches an exit, or by surviving the timeout.

So this is a zero-sum between teams but cooperative within teams, which is the core reason it's a multi-agent learning problem and not just two single-agent problems.

Environment design

The environment is built in Unity ML-Agents and structured as four scenes of increasing difficulty: an open arena, a static maze, a dynamic maze, and an unseen-evaluation maze. The dynamic maze is the principal training environment.

The dynamic part is that walls are not fixed for the whole episode. Each wall is a vertical pillar that can rise out of or sink into the floor. Every twelve seconds, the controller picks two random pillars and toggles them but only after a safety check that prevents shifting walls within 1.75 metres of any agent and prevents sealing exits.

So topology mutates during play, but never produces unfair states or unreachable goals.

Agent observations and actions

Every agent acts on a flat observation vector; 117 floats for Sentinels, 129 for Runners. The vector has five blocks: a 10-dim self-state (position, velocity, heading, alive flag, team flag); a 7-dim environment context (time remaining, nearest-exit distance and direction, wall proximity, and the wall-shift timer); an engineered 6-dim *last-known-position memory* of the tracked opponent; an omnidirectional ray sensor; 14 rays for Sentinels, 16 for Runners, with six channels each; and a 10-dim summary of the two nearest opponents in the agent's local frame.

The action space is two continuous values, each clamped to $[-1, 1]$, interpreted as a 2-D world-frame velocity in the XZ plane. The agent always faces the direction it's moving so we have continuous holonomic control on a plane, not separate forward and turn channels.

Reward design

Reward is layered into three categories.

- The *terminal* layer is a shared team reward of ± 1.0 ; every Sentinel gets +1.0 if the team captures both Runners, every Runner gets +1.0 if one escapes or they survive to timeout.
- The *shaping* layer is per-agent: chase-progress and evade-progress on the order of $1e-3$ per step, plus penalties for wall-loops, orbit-stalls, and threat approaches. This injects a per-agent gradient so individual contributions are credited even though the terminal reward is shared. It mitigates the *lazy-agent* failure mode of pure shared rewards.
- The *tactical* layer is coalition-scoped: a pincer formation rewards the two Sentinels involved, corridor control rewards the two blockers, exit denial rewards the guarding Sentinel.

Training algorithm

We train both behaviours with PPO using parameter sharing within each team. All three Sentinels share a single network; same weights, same observation schema and both Runners share a separate network. So we train two policies in total, not five.

Parameter sharing is methodologically the right partner for shared rewards. It triples our sample throughput per environment step on the Sentinel side, and it enforces role symmetry: Sentinel A and Sentinel B cannot diverge into incompatible specialisations because they are the same network. Coordination has to emerge from differing observations of the same policy, not from architecturally distinct agents.

Curriculum

Training is staged. Stage 1 is a static maze with fixed spawn points; the agent has only to learn the basic mechanics of pursuit and evasion. Stage 2 keeps the maze static but randomises spawns. Stage 3 introduces dynamic wall shifts at a slow 20-second cadence. Stage 4 increases shift frequency to 8 seconds.

Each stage has both a minimum-episode threshold and a Sentinel-win-rate gate. The agent advances only when both are satisfied so the curriculum is adaptive, not on a fixed schedule. The motivation, following Zhang et al. 2023, is that coordination behaviours are hard to discover from sparse reward in the full-difficulty environment but easier to bootstrap in simpler ones, then transfer.

Labyrinth Breach – Implementation Deep Dive

System Architecture, Code Walkthrough & Live Demo

Ahmed Jawed

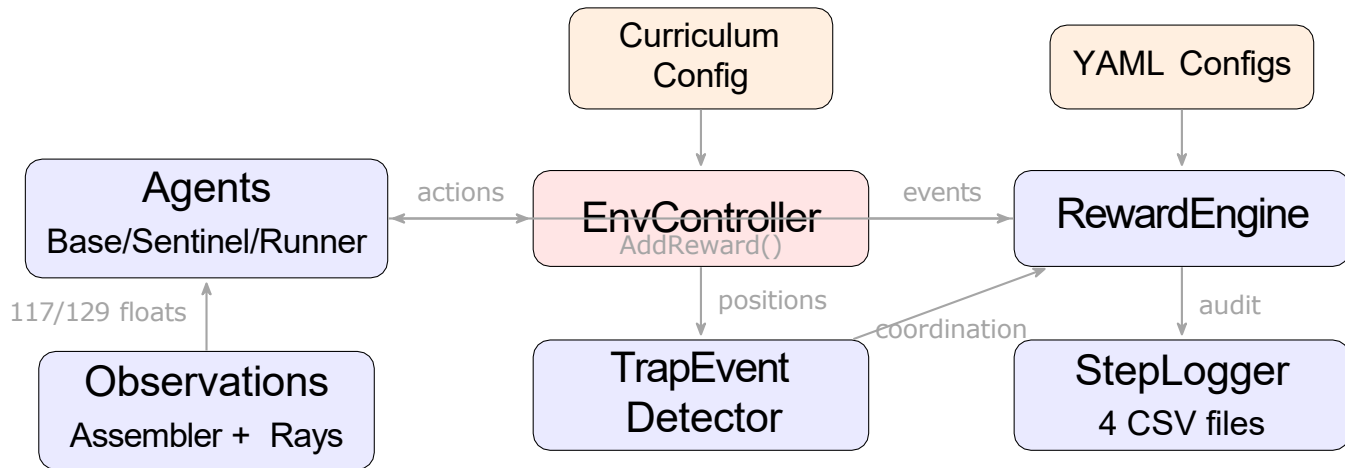
AI641 – AI for Robotics | MS AI, LUMS

May 2025

What I Will Cover

- 1 **System Architecture** – how the pieces connect
- 2 **Observation Pipeline** – how agents see the world
- 3 **Reward Engine** – how we shape behavior
- 4 **Training Pipeline** – curriculum & config flow
- 5 **Logging & Evaluation** – how we measure results
- 6 **Live Demo** – trained agents in the dynamic maze

System Architecture Overview



Key design: Everything is config-driven. Change reward weights, curriculum thresholds, or observation setup without touching C# code.

Agent Hierarchy

Inheritance Chain

- Unity.MLAgents.Agent
 - BaseAgent – movement, observation collection, ML-Agents hooks
 - SentinelAgent – capture radius, target tracking
 - RunnerAgent – escape logic, captured state

Parameter Sharing

- All 3 Sentinels → one PPO brain
- Both Runners → one PPO brain
- Same weights, different observations
- What one agent learns, all agents know

ML-Agents Lifecycle (every step)

- 1 **Observe** – CollectObservations builds the 117/129 float vector from sensors, rays, and memory
- 2 **Decide** – neural network maps observation → 2 continuous actions (moveX, moveZ)
- 3 **Act** – OnActionReceived applies movement to the agent's rigidbody
- 4 **Reward** – RewardEngine computes and assigns reward
- 5 **Log** – StepLogger records the event to CSV

Episode boundaries

- Start: agents reset position and memory

Observation Pipeline – How Agents See

Component	Sent.	Run.
Self state	10	10
Env context	7	7
Rays ($N \times 6$)	84	96
LOS memory	6	6
Opponent summary	10	10
Total	117	129

Each ray = 6 floats:

- 1 Normalized hit distance
- 2 Wall (one-hot)
- 3 Exit zone (one-hot)
- 4 Teammate (one-hot)
- 5 Opponent (one-hot)

Design Decisions

- **No camera/pixels** – vector observations train 10x faster and are fully interpretable
- **No global state** – agents never see the full maze, making this a true POSG
- **Memory** – stores last-known XYZ when target leaves LOS, decays over time
- **Opponent summary** – 2 nearest enemies: distance, relative position, visibility, threat pressure

Why more Runner rays?

Runners are prey – they need wider 360°

Reward Engine – How We Shape Behavior

Sentinel Rewards

Event	Weight
Team capture (win)	+1.0
Timeout/escape (loss)	−1.0
Individual capture	+0.1
Chase progress	+0.003
Chase regression	−0.0025
Wall loop penalty	−0.002
Exploration bonus	+0.0008

Runner Rewards

Event	Weight
Team exit (win)	+1.0
Both captured (loss)	−1.0
Survival/step	+0.0005
Eyade progress	+0.0025

Reward Pipeline

YAML config → RewardConfig (parse) →
Role-specific policy → RewardEngine (apply) →
agent.AddReward() → StepLogger (CSV audit)

Key design: Terminal rewards are ± 1.0 .
Shaping rewards are ~ 0.001 – 0.003 . This ensures
the agent always prioritizes winning over farming
intermediate bonuses.

Tactical Layer (TrapEventDetector)

- Pincer: 2 Sentinels on opposite sides
- Corridor control: both ends blocked
- Exit denial: guarding exit
- Dead-end forcing
- Cluster penalty (anti-

Training Pipeline – Curriculum & Config

4-Stage Curriculum

Stage	Ep.	Walls	WR
Static, fixed	1200	–	.62
Static, random	2200	–	.55
Dynamic, 20s	3200	20s	.50
Dynamic, 8s	3000	8s	.45

Advance rule:

Must hit BOTH min episodes AND win rate threshold before advancing.

Why this order?

- 1 Basic pursuit in open space
- 2 Prevent route memorization
- 3 Adapt to slow changes
- 4 React to fast topology shifts

PPO Hyperparameters

Parameter	Value
Learning rate	3×10^{-4}
Batch size	1024
Buffer size	10240
Hidden layers	2×256
Epochs	3
γ	0.99
λ (GAE)	0.95
ϵ (clip)	0.2
β (entropy)	0.005
Time horizon	64
Max steps	500K

Follows MAPPO recipe (Yu 2022)

Logging & Evaluation – How We Measure

4 CSV Output Files

- 1 episode_log.csv
outcome, duration, captured count
- 2 agent_step_log.csv
per-agent XYZ every step
- 3 reward_audit.csv
every reward event with type & value
- 4 replay_events.csv
captures, escapes, tactical events

Evaluation Protocol

- 3 seeds: 42, 101, 202
- Seen layouts (training mazes)
- Unseen layouts (new procedural mazes)
- Metrics averaged across seeds

Results	Metric	Seen	Unseen
	Sentinel WR	62.0%	60.3%
	1st capture	17.2s	9.7s
	2nd capture	37.8s	24.5s
	Escape frac	26.0%	26.8%
	Timeout surv	12.0%	12.9%

Key insight:

Win rates transfer (1.7pt drop).

Capture timing shifts (7.5s faster).

Policies are topology-sensitive but not layout-memorized.

Safe claim: “Policies transfer but are topology-sensitive.”

Summary – Implementation Highlights

What makes this more than a toy project:

- Modular architecture (agents, rewards, sensors, logging are all separate)
- Config-driven (YAML for everything: rewards, curriculum, PPO, evaluation)
- Full audit trail (4 CSV logs track every reward, every step, every event)
- Reproducible (3 seeds, seen/unseen split, public repo)
- Dynamic environment (walls shift during episodes)

Honest limitations:

- No MA-POCA comparison yet (credit assignment)
- No ablation study (memory on/off, shaping on/off)
- Coordination is observed qualitatively but not quantified with dedicated metrics
- BufferSensor path exists but is untested

Repo: github.com/ahmedjawedaj/labyrinth-breach-marl